理解大模型开发（一）

- 来源: https://class.imooc.com/lesson/5267#mid=104903
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第4周 理解大模型开发
- 模块: 理解大模型开发
- 课节: 图文 1-2 理解大模型开发（一）

1-2 理解大模型开发（一）

理解大模型开发（一）

> [!TIP] 阅读指南 当ChatGPT在2022年出世后，无数公司开始宣布"全面AI化"的战略，仿佛不把AI塞进每一个功能模块，就会被时代抛弃。 这让我想起十几年前云计算刚兴起时的情景。那时也是全民上云，所有系统都要迁移到云端。但最后真正用好云的企业，从来不是那些盲目追风的，而是清楚知道什么适合上云、什么不适合的公司。 大模型也是一样。 它不是魔法，不是万能药。它有擅长的事，也有做不好的事。关键在于，找准它的位置。 本章将探讨大模型在应用开发中的真实定位，帮助建立清晰的认知框架：什么场景适合用AI，什么场景应该用传统技术。 本章分为上下两篇： 上篇（本章）：三大核心能力、两种应用形态、AI定位与判断标准（认知基础） 下篇（下一章）：混合架构实战、接入方式选择、思维方式转变（实践指南）

2.1 大模型的三大核心能力

在深入讨论如何使用AI之前，需要先建立一个清晰的认知：大模型在应用开发中到底扮演什么角色?

当前大语言模型为应用开发提供了三种关键能力。

- 自然语言理解（Natural Language Understanding，NLU）

将模糊的、多样化的自然语言转换为确定的结构化信息。

```
# 输入：自然语言（模糊、多样）
input = "我想退掉昨天买的那件蓝色T恤"

# 大模型理解后输出：结构化信息（明确、可处理）
output = {
  "intent": "退货申请",
  "time": "昨天",
  "product": "T恤",
  "color": "蓝色"
}
```

这就是语义理解到指令转换的过程。传统的关键词匹配无法处理“昨天买的那件”这种灵活表达，但大模型可以。

- 内容生成（Natural Language Generation，NLG）

根据输入的数据或要求，生成符合语境的文本、代码、创意内容。

```
# 输入:结构化数据
input = {
  "name": "清透防晒霜",
  "spf": "50+",
  "feature": ["清爽", "保湿", "不油腻"]
}

# 大模型生成输出:营销文案
output = "夏日必备!清透防晒霜SPF50+,清爽质地不油腻，高倍防护同时深层保湿,让肌肤远离紫外线伤害。"
```

从数据到有温度的语言，这是大模型擅长的事。

- 逻辑推理与知识应用

基于已学习的知识，进行推理、判断和决策。下面这个场景我估计很多程序员都遇到过:

```
# 输入:程序员的求助
input = """
我写了个Python脚本,本地运行没问题,
部署到服务器就报错:FileNotFoundError: data/config.json
明明文件就在那里,好烦啊!
"""

# 大模型推理输出:分析 + 解决方案
output ="""
【问题诊断】
典型的相对路径陷阱!你遇到的是Python开发中的经典坑:

本地运行:
  你在项目根目录执行 → 相对路径正确
  ......

服务器运行:
  可能在其他目录执行 → 相对路径失效
  ......

【解决方案】
import os
........

【原理】
相对路径依赖“当前工作目录”,而工作目录会随执行位置变化。
使用 __file__ 获取脚本自身位置,再拼接相对路径......
"""
```

这是从问题描述到专业诊断和解决方案的推理能力，大模型能理解技术场景、分析原因、给出可执行的代码。

> [!TIP] 如果期望大模型解决的问题不在以上三种能力范围内，需要权衡是否真的需要大模型。很多场景下，传统技术就足够了。

2.2 AI应用的两种形态

> [!TIP] 为了行文方便，我们偶尔也会使用大模型的英文简写LLM(Large Language Model)来指代大模型；也可能会用AI(Artificial Intelligence)来描述，因为有的技术可能不只是用到了大模型，用AI泛指更合适。

理解了大模型的核心能力后，来看看AI应用的分类。AI应用可以分为两类。

LLM主导型应用

核心特征：大语言模型（LLM）是产品的灵魂，传统代码是配角。

核心特征

产品的核心价值完全由LLM能力驱动，传统技术只负责“包装”和“分发”LLM的输出。

典型代表包括对话类产品（ChatGPT智能对话助手、豆包通用AI助手）、编程辅助类（Cursor、TRAE的AI开发助手）、图像生成类（Midjourney文生图工具、Stable Diffusion开源AI绘画）、Agent系统类（自主规划任务、调用工具、执行复杂流程）。

这些应用的产品价值完全来自LLM，去掉LLM后产品就不存在。用户使用这些应用的目的就是为了获得LLM的智能能力。

架构特点包括：LLM占比80%到100%，核心功能完全依赖大模型。产品迭代跟随LLM能力升级，架构围绕LLM API构建，传统代码很薄。LLM性能直接决定产品质量，调用量巨大时API费用是主要成本。

传统主导型应用（AI辅助）

核心特征：传统业务是主体，LLM是增强插件。

产品的核心价值由传统业务逻辑提供，LLM只在特定环节发挥“智能化”作用。

典型代表包括电商平台（核心是商品/订单/支付系统，LLM只做智能客服、文案生成、个性化推荐）、在线教育平台（核心是课程内容和学习管理，LLM只做作业批改、学习路径推荐）、医疗系统（核心是病历管理和诊疗流程，LLM只做影像辅助诊断、文书生成）。

架构特点:

```
传统业务层(核心)  ← 主导者
  ├─ 用户管理
  ├─ 订单处理
  ├─ 支付结算
  └─ LLM增强模块(10-20%)
       ├─ 智能客服
       ├─ 内容生成
       └─ 数据分析
```

架构特点包括：LLM占比5%到20%，其余功能使用传统技术。产品演进由业务需求驱动，LLM是可选增强。AI调用频率低，大部分逻辑不产生API费用。核心业务不依赖LLM的不确定性，稳定性高。

> 这类应用仍以Spring/MySQL等传统框架/关系型数据库为主，LLM作为外部服务(类似外挂)在特定场景下被调用。

理解了两种形态后，如何判断应用属于哪一类？这里有一个简单法则：

简单判断法则

如果去掉LLM产品就不存在了，属于AI主导型。如果去掉LLM产品仍能运行，只是少了智能化，属于传统主导型。

例如，ChatGPT去掉GPT后什么都没了，属于LLM主导型。电商去掉智能客服后仍可用人工客服，产品照常运行，属于传统主导型。

2.3 LLM如何增强传统业务

对于LLM主导型应用（如AI写作助手），系统架构其实相对清晰：整个应用的核心就是大模型的调用，开发重点在于Prompt设计、上下文管理、工具调用等。这类应用的逻辑相对简单直接，后续章节会详细讲解。

先来探讨一个更普遍的问题：LLM如何与传统业务系统结合？

说到LLM应用开发，很多人第一反应是：“我要做一个DeepSeek那样的应用！”

但现实是，大多数开发者面临的场景并不是这样。

可能已经有一个运行良好的业务系统：

- 电商平台每天几万订单
- 在线教育平台有几十万用户

现在领导说：“我们要加入AI能力！”，应该怎么做？

很多人在这里就搞不清楚了：

- 是把LLM当成核心，重构整个系统？
- 还是把LLM当成工具，加入到现有架构？
- LLM应该放在哪个层次？
- 哪些功能适合用LLM，哪些不适合？

所以，先讨论这个问题：LLM在传统业务系统中应该扮演什么角色？

2.4 传统业务系统中的AI定位

对于传统业务系统，LLM应视为外部接入的智能服务，类似支付系统、短信服务。

这意味着，LLM不是替代系统，而是增强系统。核心业务逻辑仍是传统代码，保证确定性和可控性。LLM只在需要理解或生成时被调用，完成任务后返回结果，系统继续执行后续流程。

为什么是“外部组件”？

因为当前的AI（LLM）擅长理解和生成，但不擅长确定性计算（价格结算、库存管理）、事务处理（订单状态机、支付流程）、数据持久化（用户信息、历史记录）。

这些仍需传统技术。

2.5 常见误区：“全面AI化”的陷阱

理解了LLM的能力边界后，看一个反面案例。

假设要做一个电商网站。按照“全面AI化”的思路，可能会这样设计：

- 用AI来验证用户密码（让AI“理解”密码是否正确）
- 用AI来计算订单金额（让AI“推理”价格）
- 用AI来处理支付（让AI“生成”支付流程）

听起来很先进，但实际上，这是一场灾难。

因为这些任务的本质是确定性计算，而不是语义理解。验证密码需要的是精确的字符串匹配和加密算法，不需要"理解"。计算价格：单价10元，数量3件，总价30元，这是算术，不是推理。处理支付：验证、扣款、回调，这是状态机，不是生成。

所有的这些都不是现在AI（LLM）所擅长的。

2.6 找准AI的位置：90%传统，10%AI

在一个真实的应用系统中，AI应该用在哪里？用电商网站这个例子深入分析。

核心模块包括用户系统（注册、登录、权限）、商品系统（展示、分类、搜索）、订单系统（下单、支付、物流）等。

这些模块，90%的代码不需要AI。

它们需要的是稳定（用户每次登录，体验都应该一致）、准确（价格计算不能有0.01元的误差）、快速（查询订单状态应该在100毫秒内返回）、可靠（支付流程不能因为AI“心情不好”就失败）。

这些要求，传统技术（数据库、缓存、消息队列）做得很好，而且成本低、性能高、可控性强。

LLM应该用在哪里？

只在那些需要理解语义或生成内容的地方。

用户可能会这样问：

- “我的订单怎么还没到？”
- “这衣服会不会掉色啊？”
- “能不能换个颜色？”
- “你们送货到乡下吗？”

同一个问题，一千种问法。传统的关键词匹配处理不了，但LLM可以，它能理解自然语言的语义。

商品文案生成

有一万个商品，每个都要写吸引人的描述。人工写太慢，模板化的文案又千篇一律。LLM可以根据商品属性，生成各具特色的营销文案。

用户评论分析

每天收到几千条评论。哪些是好评？哪些是差评？哪些提到了质量问题？哪些提到了物流问题？AI可以快速分类和提取关键信息。

LLM的价值，在于处理那些没有固定规则、需要“理解”和“生成”的任务。

2.7 更宽泛的判断标准

这里总结了四个判断标准，可以快速决定一个功能是否需要LLM：

能用“if...then...”完整描述吗？

如果能，用传统技术。如果不能（规则无法穷举），考虑AI。

例如，“如果密码匹配，则登录成功”用传统技术。“如果用户问‘能退货吗’...”无法穷举所有问法，需要AI。

允许出错吗？

如果不允许，用传统技术。如果允许一定误差，可以用AI。

例如，计算订单总价绝对不能出错，用传统技术。推荐商品推荐不准也没关系，可以用AI。

需要“理解”或“生成”内容吗？

如果不需要，用传统技术。如果需要，用AI。

例如，查询库存只需读数据库，用传统技术。理解用户的自然语言提问需要AI。

调用频率和成本如何？

如果高频调用（每天百万次以上），用传统技术。如果低频调用，可以用AI。

例如，每次页面刷新都要验证登录状态用传统技术（成本几乎为零）。偶尔生成一篇营销文案用AI（调用几次无所谓）。

> [!IMPORTANT] 核心原则：能用传统技术解决的，就不用AI。 AI应该用在那些传统技术做不好的地方。 目前来说，大多数公司在使用AI能力时，都会选择调用云端API。这个成本其实是挺贵的。做个Demo玩一下，花不了几个钱；但真正的高频调用是非常烧钱的，尤其是对于技术实力不强的公司，很容易造成成本失控。 慎重、慎重，能不用AI能力就不用。

2.8 下节预告

理解了LLM的定位、能力边界和判断标准后，下一章将进入实战环节。

如何在真实系统中落地AI？智能客服的"四棒接力赛"是怎么跑的？调用API和自建模型该怎么选？从确定性编程到概率性编程，思维需要做出哪些调整？

下一节《理解大模型开发（下）- 实战篇》，我们将通过完整案例，展示AI和传统技术如何协作。

2.9 ■ 学点英语

中文	English	音标	说明
自然语言理解	Natural Language Understanding (NLU)	/ˈnætʃrəl ˈlæŋgwɪdʒ ˌʌndərˈstændɪŋ/	LLM将模糊自然语言转换为结构化信息的能力
自然语言生成	Natural Language Generation (NLG)	/ˈnætʃrəl ˈlæŋgwɪdʒ ˌdʒenəˈreɪʃn/	LLM根据输入生成符合语境文本或代码的能力
LLM主导型应用	LLM-Centric Application	/ˈlːm ˈsentrɪk ˌæplɪˈkeɪʃn/	LLM是产品灵魂，去掉LLM后产品不存在的应用形态
传统主导型应用	Traditional-Centric Application	/trəˈdɪʃənl ˈsentrɪk ˌæplɪˈkeɪʃn/	传统业务是主体，LLM是增强插件的应用形态